

The Ironclad Credentials Vault

AES-256-GCM Encryption: Zero Secret Visibility in Model Prompts

PAGE 36 / 42

Protecting Secrets Against Prompt Injection & Extraction

A major vulnerability in traditional AI wrappers is passing API keys and passwords directly inside prompt contexts, making them susceptible to prompt-injection extraction. **Fathom isolates credentials behind an AES-256-GCM encrypted vault.**

1. HARDWARE-GRADE ENCRYPTION (RING CRATE)

- Secrets stored in SQLite are encrypted using **AES-256-GCM** authenticated encryption.
- Encryption keys are derived from secure environment variables (`FATHOM_CREDENTIAL_KEY`).
- API listing endpoints return strictly masked strings (e.g. `sk-live-***1234`).

2. ZERO SECRET TOOLS IN AGENT REGISTRY

- Agents have **zero tools to read or query plaintext secrets**.
- When an agent invokes a service (e.g. CRM push), the backend adapter resolves credentials internally in Rust memory.
- Prompt injection attacks cannot extract keys that the LLM has no mechanism to access.

Credential Lifecycle Architecture

1. Operator Inputs Secret

Via UI / Protected API



2. AES-256-GCM Vault

Encrypted in SQLite DB



3. Internal Rust Resolution

Resolved by adapter at call-time



4. Zero Prompt Exposure

LLM sees zero API tokens

CRYPTOGRAPHIC SEPARATION OF CONCERNS

The AI model reasons about business concepts (e.g. *"Push lead to HubSpot"*) without ever seeing the API token. The Rust adapter injects the decrypted bearer token at the network layer just before dispatching the HTTPS request.

Enterprise Compliance: Enterprise security teams can deploy Fathom with full confidence that proprietary CRM tokens and database passwords remain completely secure.